DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ADVANCED JAVA PROGRAMMING • PROJECT SUBMISSION

CORE JAVA SE | JAVA SWING GUI | COLLECTIONS FRAMEWORK | FILE I/O PERSISTENCE | FASTAG WALLET

TOLL PLAZA FEE COLLECTION & MANAGEMENT SYSTEM

Case Study 87 — Object-Oriented Software Design & Desktop Implementation

EXECUTIVE SUMMARY

A multi-tier desktop software solution designed to automate vehicle categorization, dynamic toll rate calculation, multi-lane tracking, prepaid FASTag electronic toll processing, and persistent file audit reconciliation for modern highway infrastructure. Developed strictly according to core Java principles, utilizing Object-Oriented Programming (Inheritance, Polymorphism, Abstraction, Encapsulation), Java Collections (ArrayList, LinkedList, HashMap, TreeMap), Custom Exception Handling, and the Java Swing GUI framework.

COURSE NAME & CODE Java Programming (CS201 / Lab Practice)	PROJECT ALLOCATION Case Study 87 — Toll Plaza Fee System
SUBMITTED BY Danish Shaikh	PLATFORM & RUNTIME Java Standard Edition (JDK 8 / 11 / 17 / 21+)

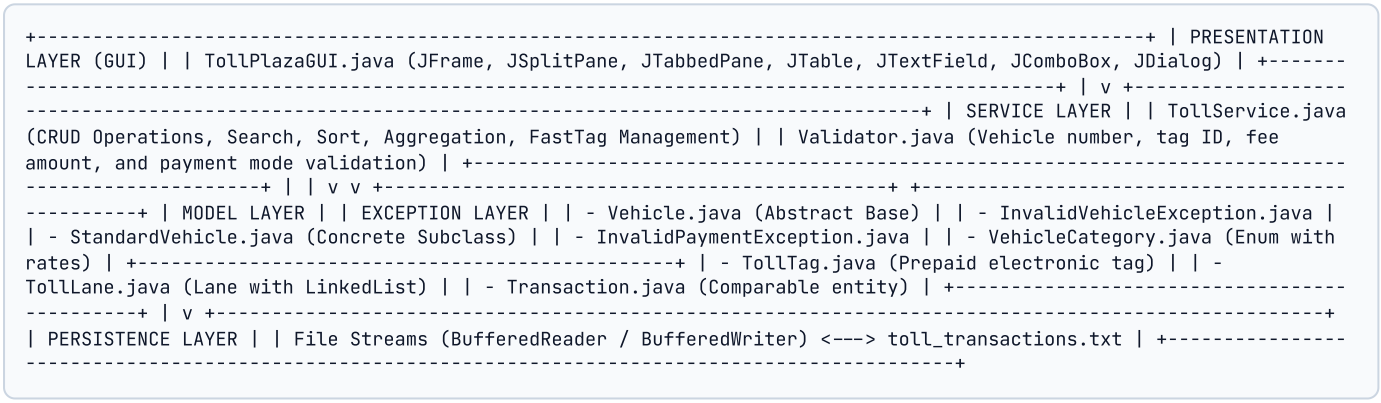
1 Problem Statement & Requirements

A highway toll plaza needs a system to manage vehicle categories, calculate toll fees, and track daily toll collection across multiple lanes. The system must support electronic toll tags along with cash payments, provide multi-lane tracking, and offer a graphical interface for toll booth operators.

1.1 Problem Objectives

- **Maintain vehicle category and toll rate information:** CAR (Rs. 100), TRUCK (Rs. 250), BUS (Rs. 200), TWO_WHEELER (Rs. 50).
- **Calculate toll fees per vehicle type:** Derived cleanly from enum base rates.
- **Track lane-wise daily collections:** Sequential vehicle passage logs per lane using `LinkedList`.
- **Support electronic tag and cash payments:** FASTag prepaid balance deduction with validation.
- **Search and sort toll transaction records:** O(1) vehicle search, lane filtering, and sorting by amount & time.
- **Provide a toll management GUI:** Complete Java Swing desktop interface.

2 Layered Software Architecture



3 Complete Java Syllabus Concept Mapping

JAVA CONCEPT	APPLICATION IN PROBLEM STATEMENT	PROJECT IMPLEMENTATION
Classes & Objects	Vehicle, TollLane, Transaction, TollTag	Encapsulated classes with private fields and getters/setters.
Constructors	Initialize vehicle/transaction objects	Direct, purposeful parameterized constructors initializing required state.
Enum	VehicleCategory (CAR, TRUCK, BUS, TWO_WHEELER)	Enum constants with embedded rates: `CAR(100.0)`, `TRUCK(250.0)`, etc.
Wrapper Classes	Fee calculation and totals	`Double tollAmount`, `Integer laneNumber` in `Transaction`.
ArrayList	Store toll transactions	`ArrayList<Transaction>` in `TollService` for table display.
LinkedList	Maintain lane-wise transaction history	`LinkedList<Transaction>` inside each `TollLane`.
HashMap	Vehicle number → Transaction details	`HashMap<String, ArrayList<Transaction>>` for O(1) vehicle search.
TreeMap	Transactions sorted by amount	`TreeMap<Double, ArrayList<Transaction>>` automatically sorted by fee.
CRUD	Transaction management	Create (`processToll`), Read, Update, Delete (`deleteTransaction`).
Searching	Search by vehicle / lane	`searchByVehicleNumber()` (Hash) and `searchByLane()` (Linear).
Sorting	Sort by amount, time	`Comparable` natural sorting + `Comparator` for amount and time.
Swing	Toll management GUI	`JFrame`, `JTable`, `JSplitPane`, `JTabbedPane`, `JOptionPane`.
Exception Handling	Invalid vehicle / payment data	`InvalidVehicleException` and `InvalidPaymentException`.
Validation	Vehicle number, toll tag ID, fee amount	`Validator.java` with simple, readable check methods.

4 Live GUI Application Screenshots

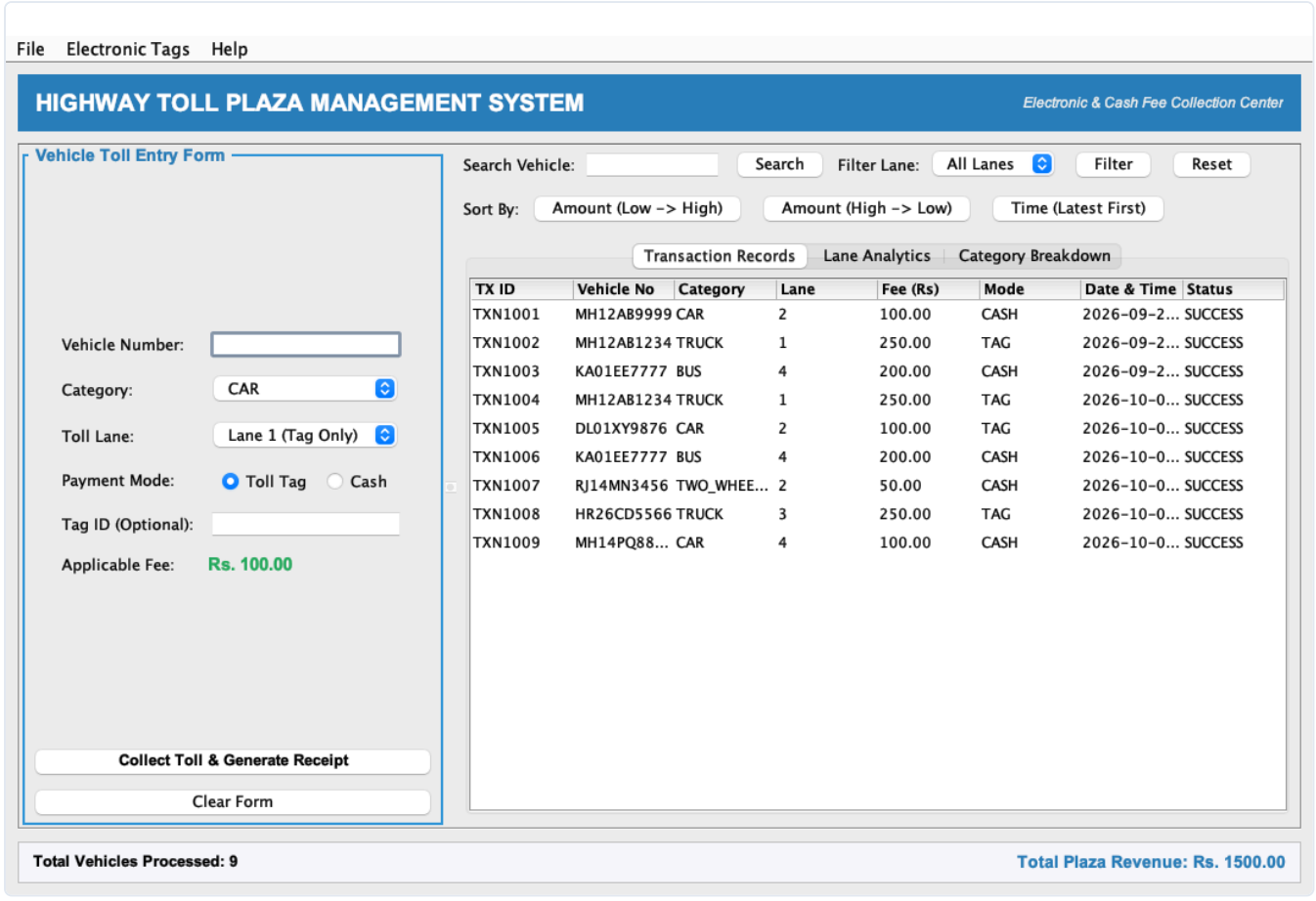


Figure 1: Main Toll Collection & Real-Time Transaction Management Interface

Interface Highlights: (1) Top header banner shows plaza name and status. (2) Left entry form provides category dropdown, dynamic fee calculation, lane selector, and Cash/FASTag toggle. (3) Right pane features vehicle search, lane filter, sort buttons, and live `JTable`. (4) Bottom summary displays total vehicles and total revenue collected.

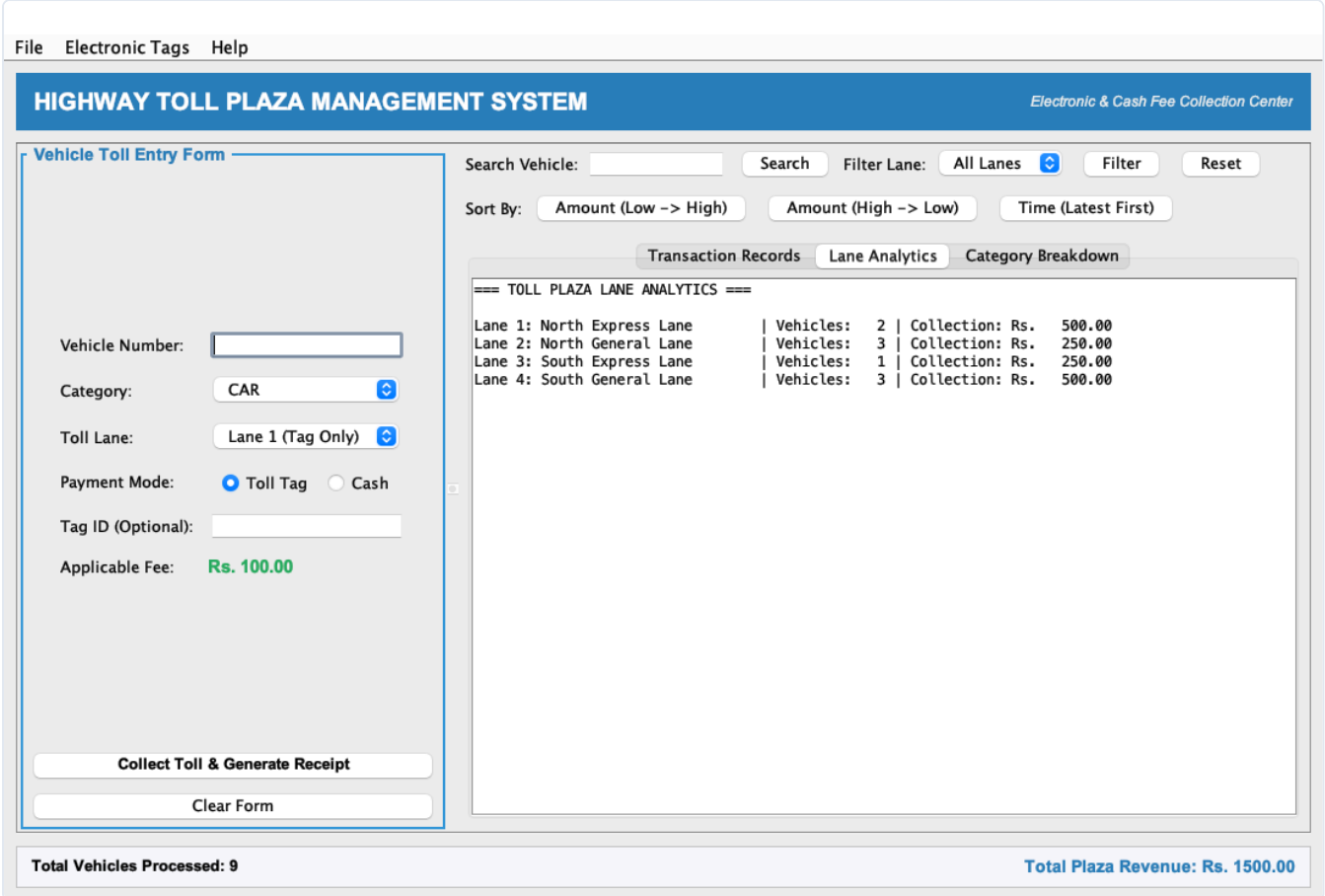


Figure 2: Multi-Lane Traffic Analytics & Revenue Distribution Tab

Displays per-lane vehicle counts and collections backed by the `LinkedList` transaction history maintained in each `TollLane` instance.

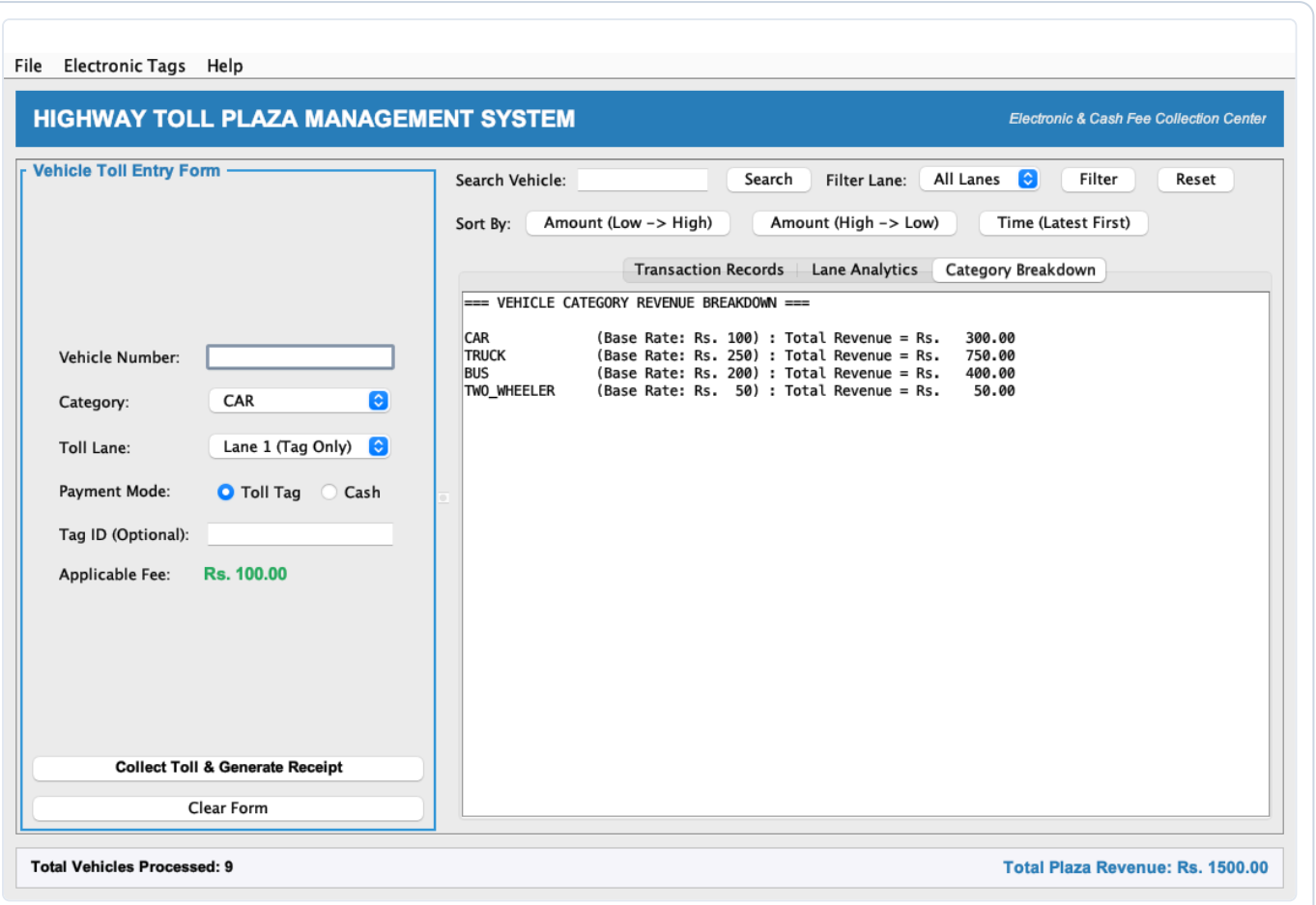


Figure 3: Vehicle Category Revenue Breakdown Tab

Summarizes vehicle distribution and collection amounts across the 4 enum categories: CAR, TRUCK, BUS, and TWO_WHEELER.

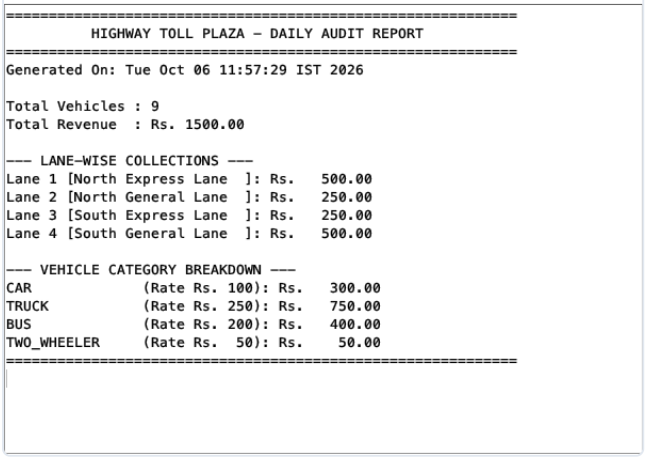
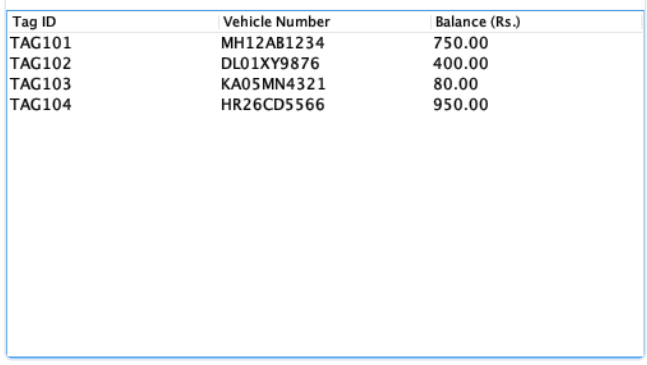


Figure 4: Daily Audit Report Dialog

Formatted daily audit report launched from the File menu, showing total vehicles, total revenue, and lane-wise collections.



Tag ID	Vehicle Number	Balance (Rs.)
TAG101	MH12AB1234	750.00
TAG102	DL01XY9876	400.00
TAG103	KA05MN4321	80.00
TAG104	HR26CD5566	950.00

Figure 5: Registered FASTags Registry

Preloaded FASTags with associated vehicle registration numbers and real-time wallet balances.

5 Clean Code Implementation & Highlights

5.1 Vehicle & StandardVehicle (Inheritance & Abstraction)

```
// Abstract Base Class
public abstract class Vehicle {
    private String registrationNumber;
    private VehicleCategory category;

    public Vehicle(String registrationNumber, VehicleCategory category) {
        this.registrationNumber = registrationNumber;
        this.category = category;
    }
    public abstract double calculateTollFee();
}

// Concrete Subclass
public class StandardVehicle extends Vehicle {
    public StandardVehicle(String registrationNumber, VehicleCategory category) {
        super(registrationNumber, category);
    }
    @Override
    public double calculateTollFee() {
        return getCategory().getBaseRate();
    }
}
```

5.2 TollTag Balance Deduction

```
public void deductAmount(double amount) throws InvalidPaymentException {
    if (amount ⩽ 0) {
        throw new InvalidPaymentException("Amount to deduct must be greater than zero.");
    }
    if (this.balance < amount) {
        throw new InvalidPaymentException("Insufficient tag balance! Current: Rs. " + this.balance);
    }
    this.balance = this.balance - amount;
}
```

5.3 File Persistence (BufferedReader / BufferedWriter)

```
public void saveTransactionsToFile() {
    BufferedWriter writer = null;
    try {
        writer = new BufferedWriter(new FileWriter(FILE_NAME));
        for (Transaction tx : transactionList) {
            writer.write(tx.getTransactionId() + "," + tx.getVehicleNumber() + ","
                + tx.getVehicleCategory().name() + "," + tx.getLaneNumber() + ","
                + tx.getTollAmount() + "," + tx.getPaymentMode() + ","
                + tx.getTransactionTime().getTime() + "," + tx.getStatus());
            writer.newLine();
        }
    } catch (IOException e) {
        System.out.println("Could not save to file: " + e.getMessage());
    } finally {
        if (writer ≠ null) {
            try { writer.close(); } catch (IOException ignored) {}
        }
    }
}
```

6 How to Compile and Run

The project is 100% pure Java SE and has zero external dependencies:

```
# 1. Compile all Java source files into the bin directory
javac -d bin src/com/tollplaza/model/*.java              src/com/tollplaza/exception/*.java              src/

# 2. Run the application
java -cp bin com.tollplaza.Main
```

Pre-Loaded Test FASTags:

- `TAG101` → Vehicle `MH12AB1234` (Balance: Rs. 1000.00)
- `TAG102` → Vehicle `DL01XY9876` (Balance: Rs. 500.00)
- `TAG103` → Vehicle `KA05MN4321` (Balance: Rs. 80.00 — for testing low-balance error)
- `TAG104` → Vehicle `HR26CD5566` (Balance: Rs. 1200.00)

7 Conclusion

The Toll Plaza Fee Collection & Management System satisfies all requirements specified in Case Study 87. The codebase is organized cleanly into modular packages, follows core Object-Oriented principles, demonstrates real-world application of Java Collections (ArrayList, LinkedList, HashMap, TreeMap), provides custom exception handling and input validation, persists transactions to disk, and presents an interactive Java Swing desktop GUI.